

1. **Entidades y atributos:** Entidad → tabla, identificador → clave primaria · Atributos compuestos y derivados · Atributo multivaluado → tabla propia
2. **Relaciones binarias:** Relación 1:N → clave ajena en el lado N · Relación M:N → tabla propia · Relación 1:1 → la clave pasa a uno de los lados · Participación → NOT NULL en la clave ajena
3. **Casos especiales:** Entidad débil → clave de la propietaria + clave parcial · Relación unaria (reflexiva) · Relación ternaria → tabla con tres FK
4. **Jerarquías:** Jerarquía: las tres estrategias · Cómo elegir una estrategia de jerarquía

## Entidades y atributos

### Entidad → tabla, identificador → clave primaria

Cada tipo de entidad se convierte en una **tabla** con el mismo nombre, y su identificador pasa a ser la **clave primaria** de la tabla.

- Regla 1: **entidad** → **tabla**, con el nombre de la entidad.
- Regla 2: **atributo** → **columna** de esa tabla.
- Regla 3: **identificador** → **clave primaria**.

#### Tablas resultantes

**Libro** (ISBN, título, año)

Subrayado: clave primaria. En cursiva con →: clave ajena. ? indica una clave ajena que puede ser NULL.

| Libro       |
|-------------|
| <u>ISBN</u> |
| título      |
| año         |

**Ejemplo.** La entidad Libro con ISBN, título y año se convierte en Libro(ISBN, título, año). Una fila por libro, y el ISBN lo identifica.

**Error frecuente.** Añadir una columna id artificial cuando la entidad ya tiene identificador. Si el modelo dice que el ISBN identifica un libro, el ISBN es la clave primaria.

### Atributos compuestos y derivados

Un atributo **compuesto** se descompone en una columna por cada parte. Un atributo **derivado** se omite, porque puede calcularse.

- **Compuesto** → sus partes se convierten en columnas; el nombre del compuesto desaparece.
- **Derivado** → sin columna; se calcula en las consultas.

#### Tablas resultantes

**Cliente** (id cliente, nombre, fecha nacimiento, calle, ciudad, código postal)

Subrayado: clave primaria. En cursiva con →: clave ajena. ? indica una clave ajena que puede ser NULL.

| Cliente                                  |
|------------------------------------------|
| <u>id cliente</u>                        |
| nombre                                   |
| fecha nacimiento                         |
| dirección (calle, ciudad, código postal) |
| /edad                                    |

**Ejemplo.** Cliente(id cliente, nombre, fecha nacimiento, dirección(calle, ciudad, código postal), /edad) se convierte en Cliente(id cliente, nombre, fecha nacimiento, calle, ciudad, código postal).

**Error frecuente.** Mantener una columna edad junto a fecha nacimiento. Al cabo de un año la edad almacenada es incorrecta, mientras que la fecha de nacimiento sigue siendo correcta.

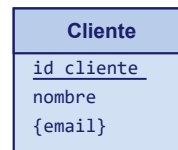
## Atributo multivaluado → tabla propia

Un atributo **multivaluado** se convierte en una tabla nueva. Su clave primaria es la clave de la entidad propietaria más el valor.

- Tabla nueva, normalmente con el nombre de la propietaria + el del atributo (ClienteEmail).
- Columnas: la clave de la propietaria (**PK y FK**) + el valor (**PK**).
- La tabla de la propietaria conserva sus demás atributos y pierde el multivaluado.

### Tablas resultantes

**Cliente** (id cliente, nombre)  
**ClienteEmail** (id cliente→Cliente, email)



Subrayado: clave primaria. En cursiva con →: clave ajena. ? indica una clave ajena que puede ser NULL.

**Ejemplo.** Cliente(id cliente, nombre, {email}) se convierte en Cliente(id cliente, nombre) y ClienteEmail(id cliente → Cliente, email).

**Error frecuente.** Añadir las columnas email1, email2, email3. Eso fija un límite arbitrario, deja NULLs y obliga a buscar un e-mail en tres sitios.

## Relaciones binarias

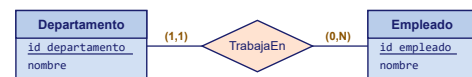
### Relación 1:N → clave ajena en el lado N

Una relación uno a muchos **no tiene tabla propia**. La clave del lado "uno" pasa al lado "muchos" como **clave ajena**.

- Regla 5: **1:N** → **FK en el lado N**, que referencia al lado 1.
- La FK es **NOT NULL** cuando el mínimo en el lado 1 es 1 (ver "Participación").

### Tablas resultantes

**Departamento** (id departamento, nombre)  
**Empleado** (id empleado, nombre, id departamento→Departamento)



Subrayado: clave primaria. En cursiva con →: clave ajena. ? indica una clave ajena que puede ser NULL.

**Ejemplo.** Departamento (1,1) — TrabajaEn — (0,N) Empleado se convierte en Empleado(id empleado, nombre, id departamento → Departamento).

**Error frecuente.** Poner la clave ajena en el lado "uno", por ejemplo un id empleado en Departamento. Solo podría guardar un empleado por departamento.

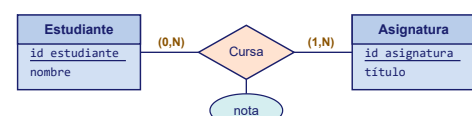
### Relación M:N → tabla propia

Una relación muchos a muchos se convierte en una **tabla nueva**. Su clave primaria combina las claves de las dos entidades, y cada una de ellas es además clave ajena.

- Regla 4: **M:N** → **tabla** con el nombre de la relación.
- PK = **las dos claves ajenas juntas**.
- Los atributos de la relación se convierten en columnas de esta tabla.

### Tablas resultantes

**Estudiante** (id estudiante, nombre)  
**Asignatura** (id asignatura, título)



**Cursa** (*id estudiante*→Estudiante, *id asignatura*→Asignatura;

Subrayado: clave primaria. En cursiva con →: clave ajena. ? indica una clave ajena que puede ser NULL.

**Ejemplo.** Estudiante (0,N) — Cursa(nota) — (1,N) Asignatura se convierte en Cursa(id estudiante → Estudiante, id asignatura → Asignatura, nota).

**Error frecuente.** Hacer clave primaria solo a una de las dos claves ajenas. Entonces un estudiante solo podría aparecer en una asignatura.

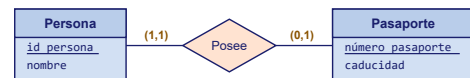
## Relación 1:1 → la clave pasa a uno de los lados

En una relación uno a uno la clave de **cualquiera** de los dos lados puede pasar al otro. Con (0,1)/(1,1), la regla del curso la pasa al lado **opcional**.

- Regla 6: 1:1 → la clave de cualquiera de los lados pasa al otro.
- Regla 7: con (0,1)/(1,1), la clave pasa al lado **opcional** (el que tiene mín 0 en el extremo opuesto).
- Usa el selector de abajo para ver los dos diseños aceptados.

### Tablas resultantes

**Persona** (*id persona*, nombre, *número*  
*pasaporte*→Pasaporte?)  
**Pasaporte** (*número pasaporte*, caducidad)



Subrayado: clave primaria. En cursiva con →: clave ajena. ? indica una clave ajena que puede ser NULL.

**Ejemplo.** Persona (1,1) — Posee — (0,1) Pasaporte se convierte en Persona(id persona, nombre, número pasaporte → Pasaporte), con una FK que admite NULL.

**Error frecuente.** Fusionar las dos entidades en una sola tabla. A veces es aceptable, pero no es la regla del curso, y deja columnas vacías para cada persona sin pasaporte.

## Participación → NOT NULL en la clave ajena

La cardinalidad **mínima** decide si una clave ajena puede ser NULL. Se lee **cruzando** la relación, en el extremo al que apunta la FK.

- Mín 1 en el lado referenciado → FK **NOT NULL**.
- Mín 0 en el lado referenciado → la FK **puede ser NULL** (se marca con ? en la notación).
- Las columnas de la clave primaria son siempre NOT NULL.

### Tablas resultantes

**Cliente** (*id cliente*, nombre)  
**Pedido** (*id pedido*, fecha, *id cliente*→Cliente, *id*  
*repartidor*→Repartidor?)  
**Repartidor** (*id repartidor*, nombre)



Subrayado: clave primaria. En cursiva con →: clave ajena. ? indica una clave ajena que puede ser NULL.

**Ejemplo.** Cliente (1,1) — Realiza — (0,N) Pedido y Repartidor (0,1) — Reparte — (0,N) Pedido se convierten en Pedido(id pedido, fecha, id cliente → Cliente NOT NULL, id repartidor → Repartidor NULL).

**Error frecuente.** Leer la cardinalidad en el lado equivocado. La cardinalidad junto a Pedido indica cuántos pedidos tiene un cliente. No decide nada sobre la FK de Pedido.

## Casos especiales

### Entidad débil → clave de la propietaria + clave parcial

Una entidad **débil** no puede identificarse por sí sola. Su clave primaria es la **clave de la entidad propietaria** (también clave ajena) más su **clave parcial**.

- PK = **clave de la propietaria (FK) + clave parcial**.
- La FK es siempre NOT NULL, porque forma parte de la PK.
- Al borrar la propietaria se suelen borrar sus filas débiles (ON DELETE CASCADE).

#### Tablas resultantes

**Edificio** (id edificio, dirección)  
**Sala** (id edificio→Edificio, número, planta)



Subrayado: clave primaria. En cursiva con →: clave ajena. ? indica una clave ajena que puede ser NULL.

**Ejemplo.** Edificio (1,1) = Contiene = (1,N) Sala(número, planta) se convierte en Sala(id edificio → Edificio, número, planta).

**Error frecuente.** Usar solo la clave parcial como clave primaria. Dos edificios con una sala 101 entrarían en conflicto.

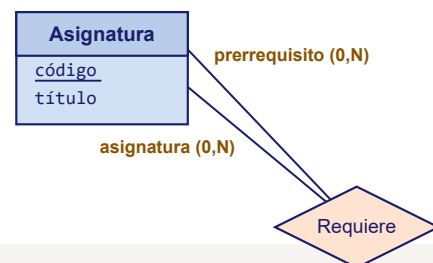
### Relación unaria (reflexiva)

Cuando una entidad se relaciona **consigo misma**, se aplica la regla habitual de 1:N o de M:N. Las claves ajenas apuntan a la **misma tabla** y reciben el nombre de los roles.

- Unaria 1:N → una FK a la misma tabla, con el nombre del rol.
- Unaria M:N → una tabla con dos FK a la entidad, una por rol.

#### Tablas resultantes

**Asignatura** (código, título)  
**Requiere** (prerrequisito→Asignatura, asignatura→Asignatura)



Subrayado: clave primaria. En cursiva con →: clave ajena. ? indica una clave ajena que puede ser NULL.

**Ejemplo.** Asignatura (0,N) como prerrequisito — Requiere — (0,N) como asignatura se convierte en Requiere(prerrequisito → Asignatura, asignatura → Asignatura).

**Error frecuente.** Crear una segunda tabla para la misma entidad, como Mentor junto a Persona. Un mentor es una persona, así que el rol es una columna, no una tabla.

### Relación ternaria → tabla con tres FK

Una relación entre **tres** entidades siempre se convierte en una tabla propia, con una clave ajena a cada una de ellas.

- Ternaria → **tabla con una FK por entidad**.
- PK: las FK de los extremos "muchos" (las tres si todos los extremos son N).

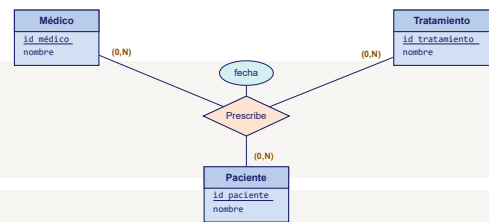
#### Tablas resultantes

**Médico** (id médico, nombre)  
**Tratamiento** (id tratamiento, nombre)  
**Paciente** (id paciente, nombre)  
**Prescribe** (id médico→Médico, id tratamiento→Tratamiento, id paciente→Paciente, fecha)

Subrayado: clave primaria. En cursiva con →: clave ajena. ? indica una clave ajena que puede ser NULL.

**Ejemplo.** Médico, Paciente, Tratamiento — Prescribe(fecha) se convierte en Prescribe(id médico → Médico, id paciente → Paciente, id tratamiento → Tratamiento, fecha).

**Error frecuente.** Dividir la ternaria en tres relaciones binarias. Se pierde qué médico prescribió qué tratamiento a qué paciente.



## Jerarquías

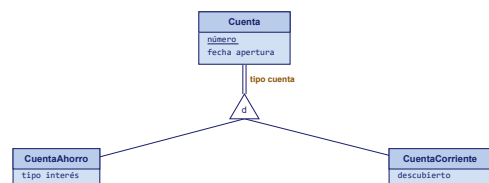
### Jerarquía: las tres estrategias

Una generalización (supertipo y subtipos) puede transformarse en tablas de **tres maneras**. Usa el selector de abajo para ver las tablas que produce cada una.

- Supertipo + subtipos: sin NULLs; leer una fila completa requiere un join.
- Una sola tabla: sin joins; NULLs en las columnas de los subtipos.
- Solo subtipos: sin joins y sin NULLs, pero no se puede referenciar el supertipo en su conjunto.

#### Tablas resultantes

**Cuenta** (número, fecha apertura)  
**CuentaAhorro** (número→Cuenta, tipo interés)  
**CuentaCorriente** (número→Cuenta, descubierto)



Subrayado: clave primaria. En cursiva con →: clave ajena. ? indica una clave ajena que puede ser NULL.

**Ejemplo.** Cuenta(número, fecha apertura) es o bien una CuentaAhorro(tipo interés) o bien una CuentaCorriente(descubierto), así que la jerarquía es total y disjunta.

**Error frecuente.** Elegir "solo subtipos" para una jerarquía parcial. Las cuentas que no son ni de ahorro ni corrientes no tendrían dónde ir.

### Cómo elegir una estrategia de jerarquía

Las tres estrategias son correctas. Elige una según cómo se **usan** los datos: relaciones compartidas, cuántos atributos propios tienen los subtipos y si la jerarquía es total.

#### Ventajas e inconvenientes de las tres estrategias

| Estrategia           | NULLs                               | Joins para leer una fila                           | Requiere jerarquía total |
|----------------------|-------------------------------------|----------------------------------------------------|--------------------------|
| Supertipo + subtipos | No                                  | Sí (supertipo ⋈ subtipo)                           | No                       |
| Una sola tabla       | Sí, en las columnas de los subtipos | No                                                 | No                       |
| Solo subtipos        | No                                  | No (pero hace falta un UNION para listarlos todos) | Sí                       |

**Error frecuente.** Omitir el discriminante en la estrategia de una sola tabla. Sin él, una fila con todas las columnas de los subtipos a NULL no puede clasificarse.